

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

Práctica 3

La jerarquía de memoria

Contexto

Hasta el momento, el alumnado ha aprendido la importancia de la serialización de los datos, así como a acceder a ellos de forma correcta. Como se verá en los próximos temas, tanto para MPI como para CUDA, etc., la serialización de los datos es de vital importancia a la hora de llevar a cabo las transferencias de estos, ya que para optimizar los envíos, siempre es recomendable pocos envíos de datos de un mayor tamaño, que no muchos envíos de reducido tamaño. Además, en la práctica 2, puso todos los conocimientos en práctica desarrollando su propio modelo de red neuronal MLP.

En esta práctica, el alumno observará el impacto del acceso a los datos almacenados en memoria. Como ya será conocedor el alumno, los servidores se componen de diferentes niveles de memoria, tal y como muestra la Figura 1.

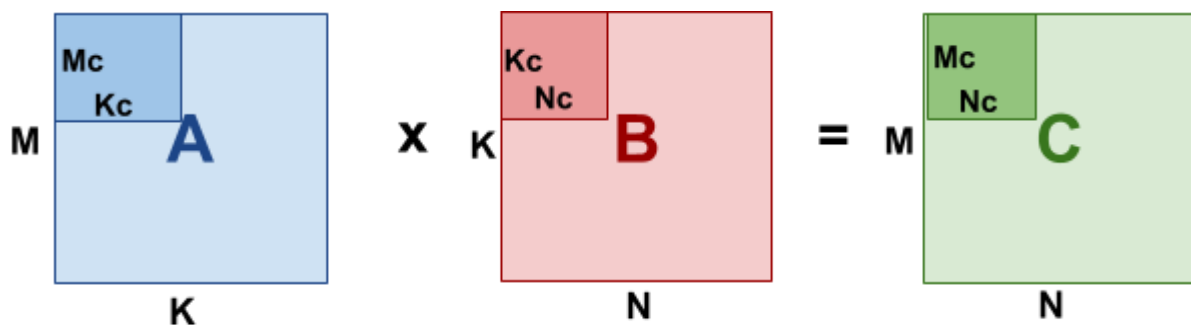


Fuente: https://es.wikipedia.org/wiki/Jerarqu%C3%ADa_de_memoria

Figura 1. Jerarquía de memoria

Tal y como se muestra en la Figura 1, conforme los datos se encuentren más cercanos a la CPU, el acceso a ellos va a ser más rápido, aumentando la velocidad de transferencia y decreciendo la latencia de acceso.

Por este motivo, las bibliotecas más usadas de álgebra lineal para el procesamiento de redes neuronales u otro tipo de cálculos (BLIS, OpenBLAS, XNNPACK, etc.), usan un esquema especial para el producto matricial basado en bloques, el cual se extiende de CPU a GPU en bibliotecas como CUBLAS. Para esto, en vez de procesar el producto matricial completo, llevando a cabo el producto completo de $M \times N \times K$ elementos, se llevan a cabo productos de $M_c \times N_c \times K_c$, tal y como muestra la Figura 2. Además, en versiones más avanzadas, para llevar a cabo el producto con instrucciones SIMD, los bloques de la matriz $M_c \times K_c$ y $K_c \times N_c$ ordenan los datos (se empaquetan) de forma que el acceso sea directo y las cargas de los valores sean secuenciales. El desarrollador siempre ha de buscar un equilibrio, ya que si el tamaño de la matriz no es elevado, puede caber en el primer nivel de caché y se desperdiciaría más tiempo llevando el empaquetado de los datos que en las ganancias obtenidas.



```

1 void Gemm(int m, int n, int k, double *A, double *B, double *C) {
2     // Declarations: mc, nc, kc,...
3     for ( jc = 0; jc < n; jc += nc )           // Loop 1
4         for ( pc = 0; pc < k; pc += kc ) {      // Loop 2
5             // B(pc : pc + kc - 1, jc : jc + nc - 1) -> Bc
6             Pack_buffer_B(kc, nc, &B(pc, jc), &Bc);
7             for ( ic = 0; ic < m; ic += mc ) {  // Loop 3
8                 // A(ic : ic + mc - 1, pc : pc + kc - 1) -> Ac
9                 Pack_buffer_A(mc, kc, &A(ic, pc), &Ac);
10                // Macro-kernel:
11                for ( jr = 0; jr < nc; jr += nr ) // Loop 4
12                    for ( ir = 0; ir < mc; ir += mr ) { // Loop 5
13                        // Micro-kernel:
14                        // Cc(ir : ir + mr - 1, jr : jr + nr - 1) +=
15                        // Ac(ir : ir + mr - 1, 1 : 1 + kc - 1) .
16                        // Bc(j, 1 : 1 + kc - 1, jr : jr + nr - 1)
17                        Gemm_kernel( mr, nr, kc, &Ac(ir, 1), &Bc(1, jr),
18                                    &Cc(ir, jr) );
19                    }
20                }
21            }
22 }

```

Figura 2. Producto matricial por bloques.

Objetivos

Esta práctica se dividirá en dos partes principales. La primera parte consistirá en que el alumnado implemente su propio test para obtener el ancho de banda a la memoria RAM y los diferentes niveles de caché. En la segunda parte, el alumno llevará a cabo una nueva implementación del producto matricial, pero esta vez por bloques, siguiendo el esquema presentado en la Figura 2 de esta práctica.

Dicho esto, los objetivos principales de la práctica son:

- Comprensión del alumnado de los diferentes niveles de memoria disponibles en un sistema.
- Impacto en el rendimiento en función de los accesos a memoria.
- Comprensión del producto matricial por bloques.

EJERCICIO 1

El objetivo de este primer ejercicio es que el alumno vea la relación entre los diferentes tipos de acceso a la memoria que se llevan a cabo en ciencia de datos y los tamaños de memoria caché del sistema.

El primer paso será obtener la información desglosada del tamaño de los diferentes niveles de memoria caché de su sistema.

Nota: Recuerdo usar la misma plataforma que en la práctica anterior y en las posteriores.

En sistemas operativos Linux, existen herramientas como **lscpu**, **lstopo (hwlock)**, etc., que nos pueden ayudar a obtener todo este tipo de información.

A continuación, rellene la siguiente tabla con la información solicitada.

Nivel de Memoria	Tamaño (MB)
L1 (datos)	
L2	
L3	
Memoria RAM	

A continuación se procederá a la implementación del test para el ancho de banda. Para tener todos la misma precisión en las mediciones, las medidas de tiempo se llevarán a cabo mediante la rutina:

```
static double now_sec(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
```

Mediante el uso de la misma, el alumno obtendrá el tiempo en segundos del fragmento de código deseado:

```
double t0 = now_sec();
//Fragmento de código
double t1 = now_sec();
```

Para llevar a cabo la implementación del test del ancho de banda y del impacto de los accesos, el alumno deberá, en primera instancia, reservar un tamaño de buffer suficientemente grande para sobrepasar el tamaño del último nivel de caché L3 (por ejemplo, 1 GB). A continuación, se implementarán diferentes rutinas, las cuales irán accediendo a los resultados de nuestro vector y operando con ellos de manera diferente.

Para esto, la primera implementación llevará a cabo un acceso secuencial a los datos e irá acumulando el contenido de cada posición del vector en una variable local "sum".

```
FOR i HASTA n
    sum += buf[i]
```

Para la segunda implementación, se creará un vector con números aleatorios (dentro del tamaño del vector que se vaya a procesar) y la rutina, en vez de acceder a valores consecutivos, accederá a la posición que indique el valor aleatorio.

```
FOR i HASTA n
    aleatorio[i] = rand() % n
...
FOR i HASTA n
    sum += buf[aleatorio[i]]
```

Nota: En el segundo caso únicamente se temporizará el acceso y acumulación (segundo bucle), no la inicialización.

Nuestro objetivo es ver el comportamiento del ancho de banda en función del tipo de acceso y del tamaño del buffer al que accedemos. Se ha indicado que la reserva es de 1 GB, pero esto únicamente es el espacio en memoria reservado. Por lo tanto, para este objetivo, se creará un bucle que irá marcando el crecimiento de 'n'. Para esto, empezaremos con tamaños de 4, 8, 16, 32, 64, 256 y 512 KB, y seguidamente, 4, 8, 16, 32, 64, 256 y 512 MB. Para cada uno de estos tamaños, se temporizará el coste de las dos rutinas de acceso por separado y se mostrará tanto el tiempo consumido como el ancho de banda. Para el cálculo del ancho de banda, el alumnado debe recordar que la fórmula responde a datos transferidos por unidad de tiempo, donde se reportarán los datos transferidos en Mb y el tiempo en segundos **Mb/s** (**Nota: ¡cuidado con las unidades!**).

Resumidamente, el alumnado deberá reportar una tabla de tiempos similar a esta para cada una de las dos implementaciones.

Datos Transferidos (Mb)	Tiempo (s)	Ancho de Banda (Mb/s)
0.004 (4KB)		
...		
512		

Recomendaciones:

- Los tiempos de acceso en muchos casos son muy pequeños, se recomienda para cada test llevar a cabo una serie de repeticiones y contabilizar todo en el tiempo. Al final, se dividirá el tiempo total entre el número de repeticiones llevadas a cabo.
- El compilador puede optimizar el código si detecta que la acumulación que se lleva a cabo internamente en los bucles no se utiliza posteriormente en el programa. Se recomienda crear una variable global:

```
static volatile uint64_t g_sink = 0;
```

Y asignar el resultado de la variable sum a dicha variable fuera de la temporización. En caso de crear rutinas independientes y devolver el valor resultante, no sería necesario.

- Se recomienda utilizar las opciones de compilación: **-O2 -std=c11**

EJERCICIO 2

Implemente el producto matriz-matriz o GEMM desarrollado en la práctica anterior siguiendo la estrategia por bloques presentada en la Figura 2.